

SNN 本地学习的逐层信息保留与损失定位

(2026-10)

摘要

本地学习算法（OTTT、NDOT、S-TLLR、TESS）通过时间局部化与空间局部化降低训练开销，但它们各自优化局部目标，没有像 BPTT 那样统一的全局梯度信号。因此在现有评估中，只能看到最终准确率，无法判断信息在哪一层损失、损失了多少。本方案提出在层与层之间引入中间量，逐层量化本地学习的信息保留程度与训练效果。具体包括三个研究方向：逐层探针、逐层表示相似度、逐层梯度对齐。三者分别从“最终效用、学习结果、学习过程”三个层面定位信息损失，共同回答“信息在哪一层损失、损失多少、为什么损失”。

本文是 2026-10 月度版本，在沿用 2026-09 核心问题与三个研究方向的基础上，补充：已完成工作、分阶段下一步计划、以及组会报告汇总。

文章汇总

RTRL: A learning algorithm for continually running fully recurrent neural networks [1];

E-prop: A solution to the learning dilemma for recurrent networks of spiking neurons [2];

OTTT: Online Training Through Time for Spiking Neural Networks [3];

NDOT: NDOT: Neuronal Dynamics-based Online Training for Spiking Neural Networks [4];

S-TLLR: S-TLLR: STDP-inspired Temporal Local Learning Rule for Spiking Neural Networks [5];

TESS: TESS: A Scalable Temporally and Spatially Local Learning Rule for Spiking Neural Networks [6].

I. 研究背景与核心问题

A. 本地学习缺少全局梯度

本地学习算法是降低 SNN 训练开销的主要途径，其核心思路是在时间维度和空间维度上切断 BPTT 的全局反向链，用局部可得的信息近似梯度。四种代表性算法在“丢什么、补什么”上依次递进：

OTTT [3] 在时间维度上丢弃脉冲介导路径（spike-mediated path），即时间 Jacobian 中经过脉冲 s_t 的 $-\lambda V_{th} D_\Psi$ 项，等价于复位路径不反传；只保留固定衰减 λ ，把时间依赖近似为 $A_t \approx \lambda I$ 。

NDOT [4] 保留时间依赖，但用神经元动力学系数 $e^l[t] = \frac{u^l[t] - V_{th} s^l[t]}{u^l[t-1] - V_{th} s^l[t-1]}$ 替代 OTTT 的固定 λ ，使时间反传系数随膜电位动态变化，比 OTTT 更接近完整 BPTT。

S-TLLR [5] 转向 STDP 风格的时间信用分配，用因果（pre-before-post）与非因果（post-before-pre）两条资格迹的瞬时组合替代显式时间反传，把时间局部性建立在突触前/后脉冲的时序关系上，而非膜电位递推。

TESS [6] 在 S-TLLR 的时间局部性基础上, 进一步用本地生成的学习信号 $m^{(l)}[t] = B^{(l)\top}(f(B^{(l)}o^{(l)}[t]) - y^*)$ 替代跨层误差反传, 同时实现时间与空间全局部。

四者的共同点是: 每层 (或每个突触) 只依据局部可得的信息优化一个局部目标, 网络内部不存在像 BPTT 那样贯穿全网的统一梯度。因此在现有评估框架下, 只能看到最终准确率, 无法回答信息在哪一层损失、损失多少。

BPTT 的梯度有明确的全局结构: 误差从输出层出发, 沿空间 (层到层) 和时间 (t 到 $t-1$) 两个方向逐层、逐时刻地传播, 最终落到每个权重上。因此对 BPTT 而言, “信息在哪一层损失”至少在梯度层面是可定义的——可以逐层检查反向传播到该层的误差信号大小。而本地学习切断了这一全局链条: 每层各自优化局部目标, 层与层之间没有统一的误差信号可追踪, 于是“信息在哪一层损失、损失多少”这一问题在现有评估框架下没有答案。

B. 核心研究问题

目前的评估方式停留在“看最终准确率”: 给每个本地学习算法在某个任务上报告一个准确率, 谁高谁好。但这无法回答三个关键问题:

- 1) 信息在哪一层损失: 本地学习相对 BPTT 的性能差距, 究竟来自浅层还是深层?
- 2) 损失了多少: 每一层各自保留了多少任务信息、丢弃了多少?
- 3) 为什么损失: 损失来自时间近似的结构性误差、空间目标替换, 还是两者耦合?

核心研究问题: 在层与层之间引入中间量, 逐层量化本地学习的信息保留程度与训练效果, 从而定位“信息在哪一层损失、损失多少”及其来源。

II. 三个研究方向

三个方向分别从不同层面切入同一个核心问题, 构成从“因”到“果”的链条: 梯度对齐度量学习过程本身是否偏离, 表示相似度度量学习结果是否被扭曲, 逐层探针度量最终任务信息是否保留。下面逐一说明每个方向要回答什么问题、为什么需要、如何定义指标, 以及与另外两个方向的关系。

A. 方向一: 逐层探针

要回答的问题: 第 l 层的表示本身携带了多少关于任务标签 y 的信息? 即“任务信息在到达第 l 层时还保留多少”。

为什么需要: 这是三个方向里最直接的“信息损失”度量。如果第 l 层的表示已经能够以高精度解码出任务标签, 说明信息在到达该层时尚未丢失; 如果解码精度很低, 说明信息在该层之前 (或该层内部) 已经丢失。逐层扫描即可定位损失发生的位置。

探针的挂法: 在每一层输出之后挂一个独立的、小的解码器 (探针), 只以该层的表示作为输入去预测任务标签。主网络的权重在挂探针时保持冻结, 探针只在冻结的表示之上训练。具体地, 第 l 层在 T 个时间步的脉冲序列 $s^l[1], \dots, s^l[T]$ 先聚合为一个向量表示 z^l , 例如取时间平均 (firing rate)

$$z^l = \frac{1}{T} \sum_{t=1}^T s^l[t] \in \mathbb{R}^{n^l}. \quad (1)$$

探针取线性分类器

$$f_{\text{probe}}^l(z^l) = \text{softmax}(P^l z^l + c^l), \quad P^l \in \mathbb{R}^{C \times n^l}, \quad c^l \in \mathbb{R}^C, \quad (2)$$

其中 C 是类别数。探针参数 P^l, c^l 通过最小化交叉熵（或其他任务损失）在训练集上拟合，训练时不回传梯度到主网络。

探针准确率的计算：在验证集上，用训练好的探针对每个样本 n 做预测 $\hat{y}_n = \arg \max_c f_{\text{probe}}^l(z_n^l)$ ，探针准确率定义为

$$\text{Acc}_{\text{probe}}^l = \frac{1}{N} \sum_{n=1}^N \mathbb{I}[\hat{y}_n = y_n]. \quad (3)$$

归一化信息保留率：为了跨层比较，把准确率归一化到“从随机猜测到输出层”的区间，定义

$$\rho^l = \frac{\text{Acc}_{\text{probe}}^l - \text{Acc}_{\text{chance}}}{\text{Acc}_{\text{probe}}^L - \text{Acc}_{\text{chance}}}, \quad (4)$$

其中 $\text{Acc}_{\text{chance}} = 1/C$ 是随机猜测水平， $\text{Acc}_{\text{probe}}^L$ 是输出层（第 L 层）探针的准确率，作为“该任务可达到的信息上限”。

如何用曲线判断信息在层间的分布：画出 ρ^l 对层索引 l 的曲线。 ρ^l 越接近 1，说明该层表示保留的任务信息越接近上限；曲线中下降最快的区间，就是信息损失最集中的层。将本地学习算法与 BPTT 的两条 ρ^l 曲线画在同一张图上：两者差距最大的层，就是该本地学习算法相对 BPTT 信息损失最大的层。

与另外两个方向的关系：探针回答“信息还在不在”，属于“最终效用”层面，但它只看结果、不解释原因——它无法说明“为什么这一层丢了信息”。要回答“为什么”，需要表示相似度（看表示结构是否被扭曲）和梯度对齐（看优化方向是否偏离）来补充。

B. 方向二：逐层表示相似度

要回答的问题：本地学习训练出的第 l 层表示，与 BPTT 训练出的第 l 层表示，在几何结构上有多像？

为什么需要：探针只能说明“信息是否还在”，但不能说明“表示本身是否被扭曲”。两个不同的表示可能保留同样的任务信息（探针准确率相同），但几何结构完全不同——例如两者相差一个复杂的非线性变换。这种结构差异虽然不降低当前任务的解码精度，却会影响泛化、迁移和与后续层的配合。因此需要直接比较表示本身，而不只是比较它们能解码出什么。

表示相似度指标的定义：对第 l 层，收集 N 个样本的表示，分别排成矩阵：本地学习给出 $Z_{\text{local}}^l \in \mathbb{R}^{N \times n^l}$ ，BPTT 给出 $Z_{\text{BPTT}}^l \in \mathbb{R}^{N \times n^l}$ ，其中第 n 行是第 n 个样本的表示 z_n^l 。比较两个表示，本质上是比较它们各自定义的“样本间几何结构”。用 Gram 矩阵刻画这一结构： $K_{\text{local}}^l = Z_{\text{local}}^l (Z_{\text{local}}^l)^\top$ 的第 (i, j) 元是样本 i 与样本 j 表示的内积，度量两样本在该表示下的相似程度； K_{BPTT}^l 同理。为了让比较不受表示均值（全局平移）的影响，先对 Gram 矩阵做中心化：记 $H = I - \frac{1}{N} \mathbf{1}\mathbf{1}^\top$ （ $\mathbf{1}$ 为全 1 向量， I 为单位阵），定义

$$\tilde{K}_{\text{local}}^l = H K_{\text{local}}^l H, \quad \tilde{K}_{\text{BPTT}}^l = H K_{\text{BPTT}}^l H. \quad (5)$$

然后定义第 l 层的表示相似度为两个中心化 Gram 矩阵的（归一化）内积：

$$\text{sim}^l = \frac{\langle \tilde{K}_{\text{local}}^l, \tilde{K}_{\text{BPTT}}^l \rangle_F}{\|\tilde{K}_{\text{local}}^l\|_F \|\tilde{K}_{\text{BPTT}}^l\|_F} = \frac{\text{tr}(\tilde{K}_{\text{local}}^l \tilde{K}_{\text{BPTT}}^l)}{\|\tilde{K}_{\text{local}}^l\|_F \|\tilde{K}_{\text{BPTT}}^l\|_F}, \quad (6)$$

其中 $\langle A, B \rangle_F = \text{tr}(A^\top B)$ 是 Frobenius 内积， $\|\cdot\|_F$ 是 Frobenius 范数。

取值范围与含义： sim^l 本质上是两个矩阵（视为高维向量）之间夹角的余弦，取值范围为 $[-1, 1]$ ，实际中通常落在 $[0, 1]$ 。 $\text{sim}^l = 1$ 表示两种表示的“样本间几何结构”完全一致（只可能相差一个保持该结构的正交变换）； sim^l 越低，表示本地学习的表示相对 BPTT 偏离越大。用 Gram 矩阵而非原始表示的原因在于：它只关心“样本之间的关系结构”，对表示自身的旋转、以及（经过归一化后）缩放不敏感，从而把比较聚焦在“结构是否被扭曲”上。

逐层曲线的读法：画出 sim^l 对 l 的曲线。浅层通常相似度较高（输入编码尚未被明显改变），越往深层差异可能越大。相似度快速下降的层，就是本地学习相对 BPTT 表示被“扭曲”最严重的层。

与另外两个方向的关系：表示相似度回答“表示结构变没变”，属于“学习结果”层面，是探针（最终效用）与梯度对齐（学习过程）之间的中间环节。它比探针更能定位“在哪一层几何结构开始偏离”，但相似度下降并不必然等于任务信息丢失——有时只是换了一个等价表示。因此需要与探针交叉验证：相似度低且探针准确率也低，才是真正“有害”的信息损失。

C. 方向三：逐层梯度对齐

要回答的问题：本地学习在第 l 层产生的梯度（或等效更新方向）与 BPTT 在第 l 层的梯度，方向是否一致？

为什么需要：探针和表示相似度都是“事后”看结果，梯度对齐看的是“学习过程”本身。即使某层当前表示与 BPTT 相似，如果本地学习的梯度方向错误，那么在后续训练中该层会逐渐偏离。梯度对齐直接度量“本地学习是否正朝着 BPTT 会走的方向优化”，因此是三个方向里最接近“原因”的度量。

梯度对齐的定义：把第 l 层的权重 $W^{l \leftarrow l-1}$ 展平为一个向量，本地学习给出的梯度记为 $g_{\text{local}}^l \in \mathbb{R}^d$ ，BPTT 给出的梯度记为 $g_{\text{BPTT}}^l \in \mathbb{R}^d$ 。二者的方向一致性用余弦相似度度量：

$$\cos^l = \frac{\langle g_{\text{local}}^l, g_{\text{BPTT}}^l \rangle}{\|g_{\text{local}}^l\| \|g_{\text{BPTT}}^l\|} \in [-1, 1]. \quad (7)$$

$\cos^l = 1$ 表示方向完全一致， $= 0$ 表示正交， < 0 表示方向相反。

平行/垂直分量分解：为了把“方向一致”进一步拆成“有效的平行部分”与“有害的垂直部分”，把 g_{local}^l 沿 g_{BPTT}^l 方向做正交分解：

$$g_{\parallel}^l = \frac{\langle g_{\text{local}}^l, g_{\text{BPTT}}^l \rangle}{\|g_{\text{BPTT}}^l\|^2} g_{\text{BPTT}}^l, \quad g_{\perp}^l = g_{\text{local}}^l - g_{\parallel}^l. \quad (8)$$

$g_{\parallel}^l$ 是本地梯度中真正沿着 BPTT 方向、能推动正确优化的分量； $g_{\perp}^l$ 是垂直于 BPTT 方向、与正确优化无关（甚至有害）的分量。相应地定义两个标量：

$$c_{\parallel}^l = \frac{\langle g_{\text{local}}^l, g_{\text{BPTT}}^l \rangle}{\|g_{\text{BPTT}}^l\|^2}, \quad R_{\perp}^l = \frac{\|g_{\perp}^l\|}{\|g_{\text{BPTT}}^l\|}. \quad (9)$$

$c_{\parallel}^l$ 是“本地梯度中有效的平行部分占 BPTT 梯度的比例”， $R_{\perp}^l$ 是“垂直偏差相对 BPTT 梯度的大小”。

逐层曲线的读法：画出 $\cos^l$ （或 $c_{\parallel}^l$ 、 $R_{\perp}^l$ ）对 l 的曲线。 $\cos^l$ 低、或 $R_{\perp}^l$ 高的层，就是本地近似在该层引入的梯度误差最大的层，也就是最可能“带偏”该层优化的位置。

与另外两个方向的关系：梯度对齐回答“优化方向对不对”，属于“学习过程”层面，是三者的“因”。梯度方向错了会导致表示偏离（方向二），进而导致任务信息丢失（方向一）。三者构成“梯度（过程）→ 表示（结果）→ 任务信息（效用）”的因果链，共同回答核心问题。

D. 三个方向的关系与整体定位

三个方向构成一条从因到果的链条，共同回答“信息在哪一层损失、损失多少、为什么损失”：

- 1) **梯度对齐**（学习过程）：定位“哪一层的优化方向被本地近似带偏”；
- 2) **表示相似度**（学习结果）：定位“哪一层的表示结构被本地学习扭曲”；
- 3) **逐层探针**（最终效用）：定位“哪一层的任务信息真正丢失”。

定位流程是：先用探针找到信息在哪一层损失（结果层），再用表示相似度确认表示在哪一层被扭曲（结构层），最后用梯度对齐解释为什么在这一层损失（原因层）。三者互补——探针只看结果、梯度只看过程，表示相似度居中承上启下——单独使用任何一个都不足以完整回答核心问题。

III. 已完成工作（2026-09 → 2026-10）

本节记录从 2026-09 到 2026-10 之间已经完成的工作，它们是下一步计划的起点。

A. 研究计划文档

确立了核心研究问题与三个研究方向（见上文），形成一份可长期迭代的研究计划。核心问题是“本地学习每层优化局部目标、无统一全局梯度，无法定位信息在哪一层损失”，三个方向（逐层探针 / 逐层表示相似度 / 逐层梯度对齐）分别从“最终效用 / 学习结果 / 学习过程”三个层面切入。

B. Project 基础代码

在 Project 目录下搭起了最小可运行基础：

- LIF 神经元（前向 Heaviside + 反向 surrogate）、多层 SNN（forward / forward_states / param_vector）；
- surrogate 接 autograd：自定义 SpikeFunction（前向 Heaviside、反向 fast-sigmoid Ψ' ），已接入 LIF.step，单样本前向 + 反向贯通；
- 数据与脉冲编码（泊松编码）、统一训练循环 train_epoch()、指标接口 evaluate()；
- forward_check.py（前向手算 + surrogate 反向贯通）、stage0_check.py（阶段 0 全链路），均跑通（PASS）；
- 尚未实现：真实 DVS 数据加载（留 TODO）。

C. PPT 笔记推导

重写了 SNNTrainingSlideNotes.tex 中 BPTT、OTTT、NDOT 三个 section:

- 统一为 Step 1-8 的详细推导，每步含完整公式 + 解释 + 符号来源 + 推导方向；
- 明确三者关系链：BPTT 给出完整时间依赖 $\epsilon^l[i] = \lambda I - \lambda V_{th} D_{\Psi,i}$ ，OTTT 丢弃复位项用固定 λI 近似，NDOT 用动态 $\epsilon^l[t]$ 替代固定 λ 。

D. 冻结的关键约定

在读论文与推导过程中冻结了若干关键约定（详见《SNN 在线本地学习：细节问题清单》）：

- LIF 复位形式：统一用形式 A， $u_{t+1} = \lambda(u_t - V_{th}s_t) + I$ ；
- 层索引：统一用 $W^{l \leftarrow l-1}$ ；
- 时间索引： $\epsilon^l[i]$ 内的 spike Jacobian 用 $D_{\Psi,i}$ ；
- 资格迹粒度：e-prop 每突触 ($O(Ln^2)$)，OTTT/NDOT/S-TLLR/TESS 每层 ($O(Ln)$)。

E. Project 文件手册与函数注释

在 Project 目录下新增 MANUAL.md，记录项目现状盘点、逐文件与逐函数说明、数学符号与代码对应表；同时为 Project 下所有 .py 文件的函数/类补齐了 docstring 与关键行内注释（不改逻辑、不改签名）。

F. 阶段 1: BPTT-SG (已完成)

手写 BPTT-SG 前向/反向并跑通收敛曲线 (python -m sanity.gradient_check 全 PASS)：

- **手写 BPTT-SG** (algorithm/bptt_sg.py)：bptt_sg_forward (纯数值前向返回每层 u, s)、bptt_sg_backward (手写反向递推返回每层 dW/db)、bptt_sg_step (前向 + 损失 + 反向)。支持 detached-reset (默认, $A_t = \lambda I$) 与 full ($A_t = \lambda I - \lambda V_{th} D_{\Psi}$) 两种模式；
- **Gate A/B/C 梯度验证** (sanity/gradient_check.py)：Gate A 验时间 Jacobian、Gate B 验空间 Jacobian、Gate C 验最终梯度，手写 vs autograd 相对误差 $< 10^{-5}$ (实测 $10^{-7} \sim 10^{-8}$)，detached 与 full 各一遍；
- **收敛曲线**：full BPTT 在 256 样本合成数据上 last3 均值 > 0.85 ；detached-reset 作对照；
- **符号对照备忘录**：新增 SYMBOL_MAPPING.md，统一论文记法 ($W^{l \leftarrow l-1}$, 1-based) 与代码记法 (weights[l], 0-based) 差 1 层的映射，修正 MANUAL.md 符号表偏移；
- **接口扩展**：LIF.step / MLPSNN.forward / forward_states 加 detach_reset 参数 (默认 False, 向后兼容)。

IV. 下一步计划

按阶段推进，每阶段有明确目标与任务。核心原则：先搭架子，架子上只挂一个能跑穿的东西 (BPTT-SG)，再逐个挂载其他算法；探针、CKA、梯度对齐的钩子必须在阶段 2 焊上。

A. 阶段 0: 搭建实验骨架

目标: 搭一套通用实验框架, 架子上只挂一个能跑到头 (前向 + 反向贯通) 的东西。(本阶段已基本完成, 见“已完成工作”; 仅真实 DVS 数据加载留 TODO)

任务:

- 1) 数据与脉冲编码模块 (DVS Gesture 为主, CIFAR10-DVS 为辅, $T = 10$);
- 2) 统一网络骨架 `SNN(nn.Module)`, 支持返回每层 u, s 与固定参数顺序;
- 3) 统一训练循环 `train_epoch()`, 支持 Accumulated / Online 两种模式;
- 4) 指标接口 `evaluate()`, 预留探针、CKA、梯度对齐的调用点;
- 5) surrogate 接 autograd: 自定义 `autograd.Function` (前向 Heaviside、反向 Ψ'), 让单样本前向 + 反向贯通。(已完成)

B. 阶段 1: 挂 BPTT-SG, 跑穿上限 (已完成)

目标: 把 BPTT-SG 作为唯一算法插入骨架, 跑出一条完整收敛曲线。

状态: 已达成。手写 BPTT-SG 前向/反向 + Gate A/B/C 梯度验证 (相对误差 $< 10^{-5}$) + 收敛曲线 (合成数据 last3 均值 > 0.85) 全部 PASS (详见「已完成工作 → 阶段 1」)。

任务:

- 1) 通过 `gradcheck` (Gate A/B/C) 验证手写 BPTT-SG 梯度与 autograd 一致 (相对误差 $< 10^{-5}$)——首任务, 先把梯度算对; (已完成)
- 2) 手写 BPTT-SG 前向与反向, 用形式 A, 支持 detached-reset 与 full 两种模式 (复用阶段 0 已接好的 surrogate); (已完成)
- 3) 跑通 DVS Gesture, 记录每层 u, s 与梯度。(留待阶段 5 前接入真实数据)

C. 阶段 2: 焊上探针、CKA、梯度对齐的钩子

目标: 在写其他算法前, 先把测量工具装好。

任务:

- 1) 逐层探针模块: 每层挂独立线性探针, 不参与主网络训练;
- 2) 逐层 CKA 模块: 收集每层表示矩阵, 计算与 BPTT 的 CKA;
- 3) 逐层梯度对齐模块: 收集每层梯度, 计算与 BPTT 的余弦相似度;
- 4) 在 BPTT 自身上验证工具本身: $CKA = 1, \cos = 1$ 。

D. 阶段 3: 挂载 OTTT / NDOT / e-prop

目标: 同一框架下逐个挂载其他算法。

任务:

- 1) OTTT: 实现 $\hat{a}^{l-1}[t]$ 前向递推与瞬时梯度, 验证递推与闭式一致;
- 2) NDOT: 实现动态系数 $e^l[t]$ 与动态迹递推, 注意 $t = 1$ 特例;
- 3) e-prop: 实现 ideal factorization 与 eligibility 局部递推, 验证 $\frac{dE}{dW_{ji}} = \sum_t L_j^t e_{ji}^t$ 。

E. 阶段 4: 接入 S-TLLR / TESS

目标: 外部验证, 优先使用作者公开代码。

任务:

- 1) 下载 S-TLLR 与 TESS 作者代码, 在统一数据下复现论文报告的数字;
- 2) 逐层测量三个指标, 确保测量口径一致。

F. 阶段 5: 逐层评估实验

目标: 回答核心研究问题。

任务:

- 1) 逐层画三张图: 探针准确率、CKA、梯度对齐;
- 2) 归因分析: temporal 近似 (OTTT vs NDOT)、spatial 局部化 (S-TLLR vs TESS)、目标替换 (BPTT vs TESS);
- 3) Pareto 分析: Memory-Fidelity、Time-Fidelity、Communication-Fidelity 三张散点图。

G. 阶段 6: 与硬件方向对接

目标: 把逐层信息分布整理成硬件可参考的规格。

任务:

- 1) 状态清单: 每个算法需存储哪些状态、状态生命周期;
- 2) 通信清单: 每层通信需求、哪些可本地化;
- 3) 更新清单: 更新频率、更新模式、权重写入次数;
- 4) 与硬件同学讨论, 定期反馈可行性。

H. 长期方向: 基于逐层定位的算法改进

定位: 本节不是当前研究计划的阶段 (不编入阶段 0-6), 而是阶段 6 之后、依赖逐层评估结论才成立的长期方向, 作为后续文章的种子。

前提: 阶段 2 焊好探针 / CKA / 梯度对齐三个测量工具、阶段 3-5 完成各本地学习算法的逐层评估之后, 才有依据开展。

落点: 用三个指标共同定位「某层的主要损失来源」——逐层探针找信息在哪一层丢失、逐层 CKA 看该层表示相对 BPTT 是否被扭曲、逐层梯度对齐看该层优化方向是否偏离; 一旦确定损失集中在第 l 层、且主要来自时间近似或空间局部化, 就针对该层改进 OTTT / NDOT 的局部近似 (例如修正该层的资格迹递推或学习信号), 而非空泛地提出新算法。

V. 组会报告汇总

本节按日期记录每次组会, 格式统一为“简述 + 关键结论 + 待讨论问题”。

A. 2026-10-08 组会

汇报主题：SNN 本地学习的逐层信息保留与损失定位——研究计划与阶段进展。

本节为本次组会的完整讲稿，逐页给出「页标题 / 幻灯片内容 / 口播要点」，可直接照着做 PPT 与口头汇报。本次覆盖：研究背景与核心问题、三个研究方向、阶段 0-1 已完成工作、下一步计划，以及一个后续方向。

第 1 页·封面

- 幻灯片：题目「SNN 本地学习的逐层信息保留与损失定位」，副标题「研究计划与阶段进展」，汇报人、日期（2026-10-08）。
- 口播：主题：SNN 本地学习的逐层信息保留与损失定位；内容：研究计划 + 阶段 0/1 已完成进展。

第 2 页·核心问题：本地学习看不到「哪一层损失」

- 幻灯片：左侧列 OTTT / NDOT / S-TLLR / TESS 四个本地学习算法，右侧画 BPTT 贯穿全网的全局梯度链，中间用「断裂」示意本地学习切断了这条链；底部一句话核心问题。
- 口播：四个本地学习算法：每层只按局部信息优化局部目标，无全局梯度链；后果：现有评估只能报最终准确率，无法回答在哪层损失、损失多少、为什么损失；本研究切入点：在层间引入中间量，逐层量化。

第 3 页·三个研究方向：三把尺子

- 幻灯片：三块并列——逐层探针（量任务信息还在不在）、逐层表示相似度 CKA（量表示几何有没有被扭曲）、逐层梯度对齐（量优化方向偏没偏）；下方一条因果链「梯度（过程）→ 表示（结果）→ 任务信息（效用）」。
- 口播：逐层探针：量任务信息还保留多少（效用）；逐层 CKA：量表示几何相对 BPTT 是否被扭曲（结果）；逐层梯度对齐：量优化方向是否偏离（过程）；三者构成「梯度 → 表示 → 任务信息」因果链。

第 4 页·整体路线图（阶段 0-6）

- 幻灯片：横向流程「阶段 0 搭骨架 → 阶段 1 跑穿 BPTT-SG → 阶段 2 焊测量钩子 → 阶段 3 挂 OTTT/NDOT/e-prop → 阶段 4 接 S-TLLR/TESS → 阶段 5 逐层评估 → 阶段 6 硬件对接」，阶段 0/1 打勾、2-6 灰显。
- 口播：六阶段：0 搭骨架 → 1 跑穿 BPTT-SG → 2 焊钩子 → 3 挂 OTTT/NDOT/e-prop → 4 接 S-TLLR/TESS → 5 逐层评估 → 6 硬件对接；阶段 0/1 已完成，本次重点讲 0/1。

第 5 页·阶段 0：把实验框架搭起来

- 幻灯片：模块图 snn/（LIF 前向 + surrogate 反向）、network/（多层 SNN）、data/（泊松编码 + 合成数据）、utils/（训练循环 + 指标接口）、sanity/（验证）；右下角「全链路 PASS」。
- 口播：LIF（前向 + surrogate 反向）、多层 SNN、泊松编码 + 合成数据、训练循环、指标接口齐备；sanity/stage0_check.py 全链路 PASS；意义：后续算法在同一框架跑、口径一致。

第 6 页·阶段 1：手写 BPTT-SG，把反向跑穿

- 幻灯片：反向递推公式——输出层 $\delta^L[t] = dL/ds_{out}[t] \odot \Psi' + \lambda \delta^L[t+1]$ ；中间层 $\delta^l[t] = ((W^{l+1})^\top \delta^{l+1}[t]) \odot \Psi' + \lambda \delta^l[t+1]$ ；权重梯度 $dW^l = \sum_t \delta^l[t] \otimes s^{l-1}[t]$ ；标注两种模式 detached-reset ($A_t = \lambda I$) 与 full ($A_t = \lambda I - \lambda V_{th} D_\Psi$)。

- 口播：纯手写前向 + 反向递推，不依赖 autograd；同时实现 detached-reset ($A_t = \lambda I$) 与 full ($A_t = \lambda I - \lambda V_{th} D_\Psi$) 两模式；目的：阶段 3 挂 OTTT/NDOT 时能逐项对照。

第 7 页·阶段 1 验证：Gate A/B/C 梯度对上了

- 幻灯片：三行表——Gate A 时间 Jacobian A_t (detached = λI / full = $\lambda I - \lambda V_{th} D_\Psi$)；Gate B 空间 Jacobian $W D_\Psi$ ；Gate C 最终梯度 dW/db (多层多步)；右侧判据「手写 vs autograd 相对误差 $< 10^{-5}$ 」，实测 $10^{-7} \sim 10^{-8}$ 。
- 口播：三道 Gate：时间 Jacobian、空间 Jacobian、最终梯度 dW/db ；手写 vs autograd 相对误差 $< 10^{-5}$ (实测 $10^{-7} \sim 10^{-8}$)；结论：手写反向与自动微分逐项一致。

第 8 页·阶段 1 收敛：手写梯度真的能训练

- 幻灯片：full BPTT 收敛数据——网络 8-64-3、 $T = 10$ 、256 样本合成数据、Adam lr= 10^{-2} 、80 epochs，最后 3 个 epoch 准确率均值 0.862 (方差 1.5×10^{-4} ， > 0.85)；detached-reset 对照 0.872。
- 口播：设置：8-64-3、 $T = 10$ 、256 样本、Adam lr= 10^{-2} 、80 epochs；full BPTT last3 均值 0.862 (> 0.85)，方差 1.5×10^{-4} ；结论：手写梯度能驱动训练收敛。

第 9 页·一个伏笔：丢复位项的代价

- 幻灯片：256 样本：full 0.862 / detached 0.872 (接近)；64 样本 (单次实验观察，未统计)：detached 降至 ≈ 0.81 、低于 full——数据难分时丢复位项的代价才显现。
- 口播：detached-reset 丢复位项 $\rightarrow A_t = \lambda I$ ，即 OTTT 的时间近似；256 样本两者接近 (full 0.862 / detached 0.872)；单次实验观察 (64 样本，未统计)：detached 降至约 0.81、低于 full——丢复位项的系统性代价；留作阶段 3 对比 OTTT 的参照。

第 10 页·冻结的约定：让后面不再重推符号

- 幻灯片：约定汇总简表——LIF 复位形式 A ($u_{t+1} = \lambda(u_t - V_{th} s_t) + I$)；超参 $\lambda = 0.5$ 、 $V_{th} = 1.0$ 、 $\beta = 4.0$ ；符号对照 SYMBOL_MAPPING.md (论文 $W^{l \leftarrow l-1} \leftrightarrow$ 代码 `weights[l-1]`，差 1 层)。
- 口播：统一 LIF 复位形式 A (与 OTTT/NDOT/e-prop 符号最接近)；超参冻结： $\lambda = 0.5$ 、 $V_{th} = 1.0$ 、 $\beta = 4.0$ ；符号对照 SYMBOL_MAPPING.md：论文 $W^{l \leftarrow l-1}$ 对应代码 `weights[l-1]` (差 1 层)。

第 11 页·下一步：阶段 2 起

- 幻灯片：阶段 2-6 清单——阶段 2 焊探针/CKA/梯度对齐钩子 (并在 BPTT 自身上验证 $CKA = 1$ 、 $\cos = 1$)；阶段 3 挂 OTTT/NDOT/e-prop；阶段 4 接 S-TLLR/TESS；阶段 5 逐层评估三张图 + 归因 + Pareto；阶段 6 硬件对接。
- 口播：阶段 2：焊探针/CKA/梯度对齐钩子，先在 BPTT 自身上验证 $CKA = 1$ 、 $\cos = 1$ ；阶段 3-5：挂各算法、逐层评估；阶段 6：整理成硬件可参考规格。

第 12 页·需要讨论的问题

- 幻灯片：4 个问题——(1) 逐层探针的准确率曲线是否真能定位信息损失 (遗留疑问 H)；(2) 逐层 CKA 与梯度对齐是否一致 (遗留疑问 I)；(3) 信息损失与资源节省的 Pareto 前沿形态 (遗留疑问 J)；(4) 精读原文遗留疑问：OTTT Theorem 1 收敛条件、NDOT clamp 逻辑、DFA 收敛性证明、TESS B 矩阵准正交性 (遗留疑问 A/B/C/D)。

- 口播：探针曲线能否定位信息损失（H）；CKA 与梯度对齐结论是否一致（I）；Pareto 前沿形态（J）；精读原文遗留：OTTT Theorem 1、NDOT clamp、DFA 收敛证明、TESS B 准正交。

第 13 页·后续方向：基于逐层定位的算法改进

- 幻灯片：逻辑链「逐层探针 / CKA / 梯度对齐 → 定位某层主要损失来源 → 针对该层改 OTTT/NDOT 的局部近似」，标注「后续方向 / 下一篇文章的切入点」。
- 口播：定位：非当前项目交付，后续文章种子；前提：阶段 2–5 逐层评估完成后才有依据；落点：探针/CKA/梯度对齐锁定某层损失来源，针对该层改 OTTT/NDOT 局部近似。

B. 2026-11-XX 组会

参考文献

- [1] R. J. Williams and D. Zipser, “A learning algorithm for continually running fully recurrent neural networks,” *Neural computation*, vol. 1, no. 2, pp. 270–280, 1989.
- [2] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, no. 1, p. 3625, Jul. 2020. [Online]. Available: <https://doi.org/10.1038/s41467-020-17236-y>
- [3] M. Xiao, Q. Meng, Z. Zhang, D. He, and Z. Lin, “Online Training Through Time for Spiking Neural Networks,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 20 717–20 730. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/82846e19e6d42ebfd4ace4361def29ae-Paper-Conference.pdf
- [4] H. Jiang, G. D. Masi, H. Xiong, and B. Gu, “NDOT: Neuronal Dynamics-based Online Training for Spiking Neural...” [Online]. Available: <https://openreview.net/forum?id=eIF0QoBSFV>
- [5] M. P. E. Apolinario and K. Roy, “S-TLLR: STDP-inspired Temporal Local Learning Rule for Spiking Neural Networks,” Oct. 2024, arXiv:2306.15220 [cs]. [Online]. Available: <http://arxiv.org/abs/2306.15220>
- [6] M. P. E. Apolinario, K. Roy, and C. Frenkel, “TESS: A Scalable Temporally and Spatially Local Learning Rule for Spiking Neural Networks,” in *2025 International Joint Conference on Neural Networks (IJCNN)*, Jun. 2025, pp. 1–9, iSSN: 2161-4407. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11227652>